SMART TICKETING AND RESERVATION SYSTEM FOR BUSES - STAR -

Prepared by Company B
Date: April 4, 2026

CLIENT APPROVAL

By signing this document, the Head of Company A formally approves the Software Requirements Specification (SRS) as the binding contractual reference for the Smart Bus Ticketing and Reservation System project. This approval confirms agreement to the scope, requirements, and deliverables outlined herein, and authorizes the development team to proceed with design, implementation, and deployment in accordance with the terms of the web contract.

Company A Head :

Signature: ..... Name: Himasha Devmina Date: 2026/04/04

Representative 1 :

Signature: ..... Name: Manusha Thivankara Date: 2026/04/04

Representative 2 :

Signature: ..... Name: Udana Athukorala Date: 2026/04/04

Contents

1 Revised SRS **3**
1.1 Introduction . . . 3
1.2 Stakeholders & Roles . . . 3
1.3 Functional Requirements . . . 4
1.4 Non-Functional Requirements . . . 4

2 Task Allocation and Workload Breakdown **5**

3 Gantt Chart **5**

4 ER Diagram **6**

5 Full Class Diagram **7**

6 Sequence Diagram **8**

7 Swimlane Diagram **9**

8 Activity Diagram **10**

9 System Architecture Diagram **11**

10 Package Diagram **11**

11 Deployment Diagram **12**

12 Graphical User Interface (GUI) Plan **13**
12.1 Page Structure and Routing . . . 13
12.2 Styling and Responsiveness . . . 13
12.3 Development Tools and Libraries . . . 13

13 Test and Deployment Plan **14**
13.1 Testing Strategy . . . 14
13.2 Deployment Pipeline . . . 14
13.3 Test Cases . . . 15

14 Git Repository **16**

15 Potential risks during the project. **16**

16 Actions to Minimize Those Risks **16**

1 Revised SRS

1.1 Introduction

- This Software Requirements Specification (SRS) and Design Document outlines the functional, non-functional and architectural specifications for the Smart Ticketing and Reservation System (STAR) for buses. The goal of STAR is to digitize reservations for long distance buses using a prepaid wallet mechanism ensuring secure and reliable booking.
- Scope covered from the proposed system.
  - Booking tickets for predefined routes based on distance and sectioning.
  - Managing a digital prepaid wallet for all transactions.
  - Viewing analytics dashboards for Passengers, Drivers, Bus Owners, and Admins.
  - Implementing automated refund handling and ticket disbursement.
- Aspects out of the proposed system's scope due to infeasibility.
  - Real-time GPS tracking of buses.
  - Support for undefined custom routes or ad-hoc short-distance passengers.
  - Mobile native application (web-only system).

1.2 Stakeholders & Roles

1. End Users
   - Passengers: Book tickets, top-up wallet, submit refund requests.
   - Drivers: View passenger manifest and verify bookings.
   - Bus Owners: View earnings, utilization, performance analytics.
   - Administrators: Manage routes, users, disputes and refunds.
2. Developers
   - Client: Defines business logic and validates deliverables.
   - Development Team: Responsible for design, implementation, testing, deployment.

1.3 Functional Requirements

Ticket Provisioning

- System calculates ticket prices based on distance/sections.
- Issues unique digital tickets tied to user ID.

Prepaid Wallet

- Supports wallet top-up via payment gateway.
- Handles fare deductions and automated refunds.

Unique ID Booking

- Tickets must be linked to a unique customer ID.
- System must prevent duplicate bookings and ticket scalping.

Role-Based Dashboards

- Passenger Portal
- Driver Portal
- Owner Dashboard
- Administrator Control Panel

1.4 Non-Functional Requirements

Refund Policy

- Robust automated refund decision flow.
- Manual override by administrators.

Scalability

- Must support regional deployment initially, with island-wide scalability.

Availability

- 99.9% uptime.
- 24/7 access for bookings and modifications.

2 Task Allocation and Workload Breakdown

Module	Responsibility	Lead-Employee
Admin and Driver module	Frontend utilities : Dashboard and interfaces	Inuka
Admin and Driver module	Backend setup with analytics	Deelaka
Home Page	Frontend utilities : Dashboard and interfaces	Kushen
Home Page	Backend setup and integrating all modules	Lakindu
Passenger module	Front end utilities : Dashboard and interfaces	Dulith
Passenger module	Backend setup for passenger, wallet and Bus routing	Lasini
Owner module	Front end utilities : Dashboard and interfaces	Sandul
Owner module	Backend setup	Rumeth

Table 1: Task Allocation.

3 Gantt Chart

The planned timeline for the project is as follows. The complete project was broken down into modules and forecasted as shown.

Link: Gantt chart

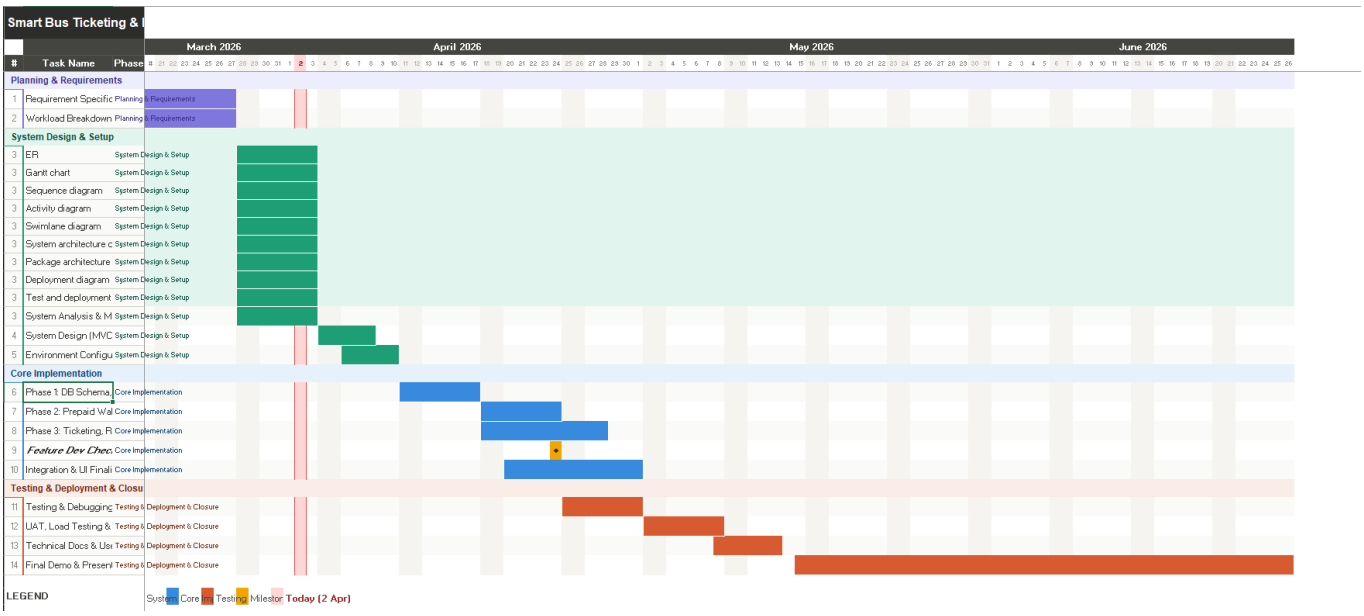

Figure 1: Gantt chart

4 ER Diagram

The ER diagram contains all the relationships between the entities proposed in the system architecture.

Link: ER Diagram

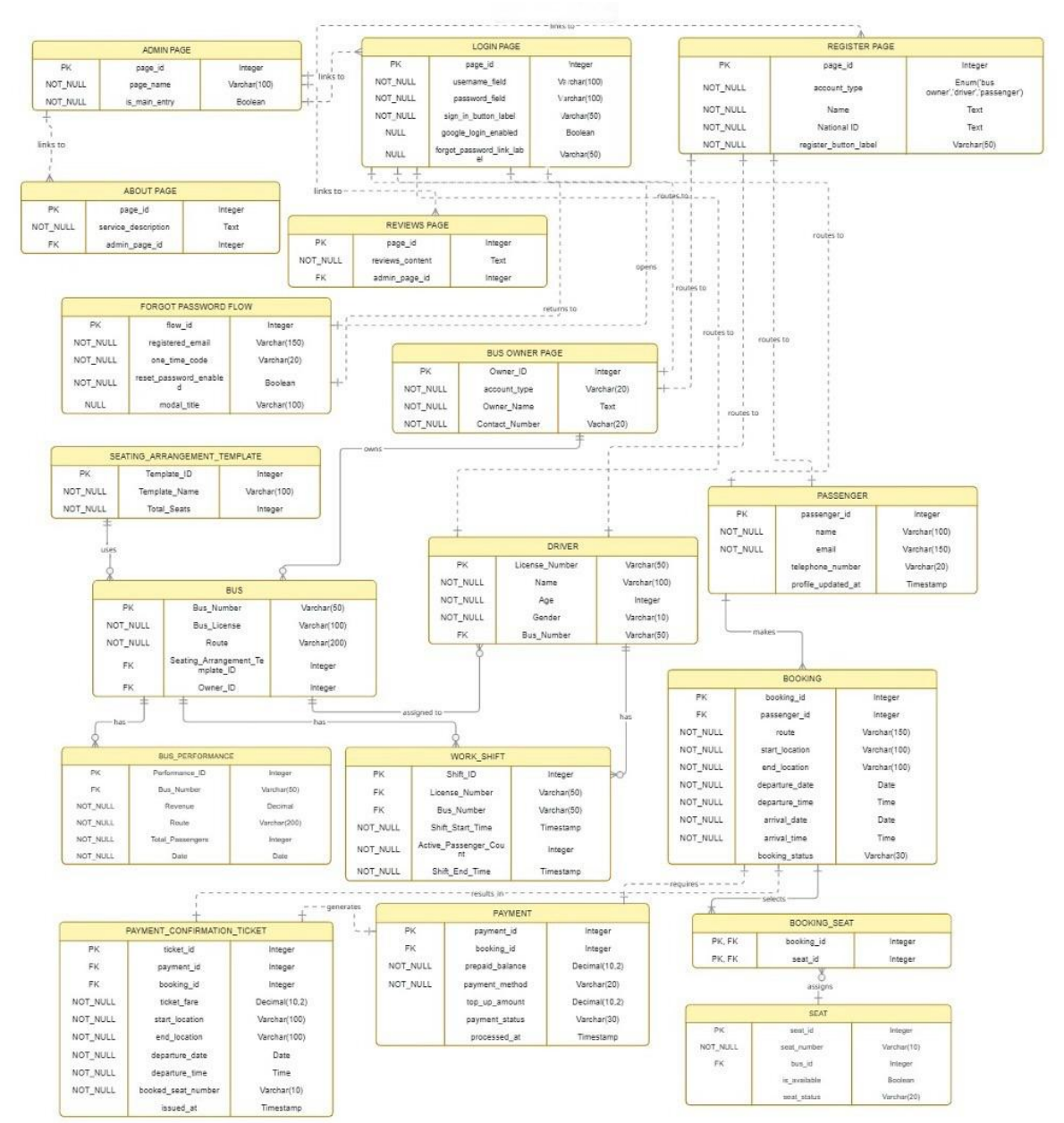

Figure 2: ER diagram

5 Full Class Diagram

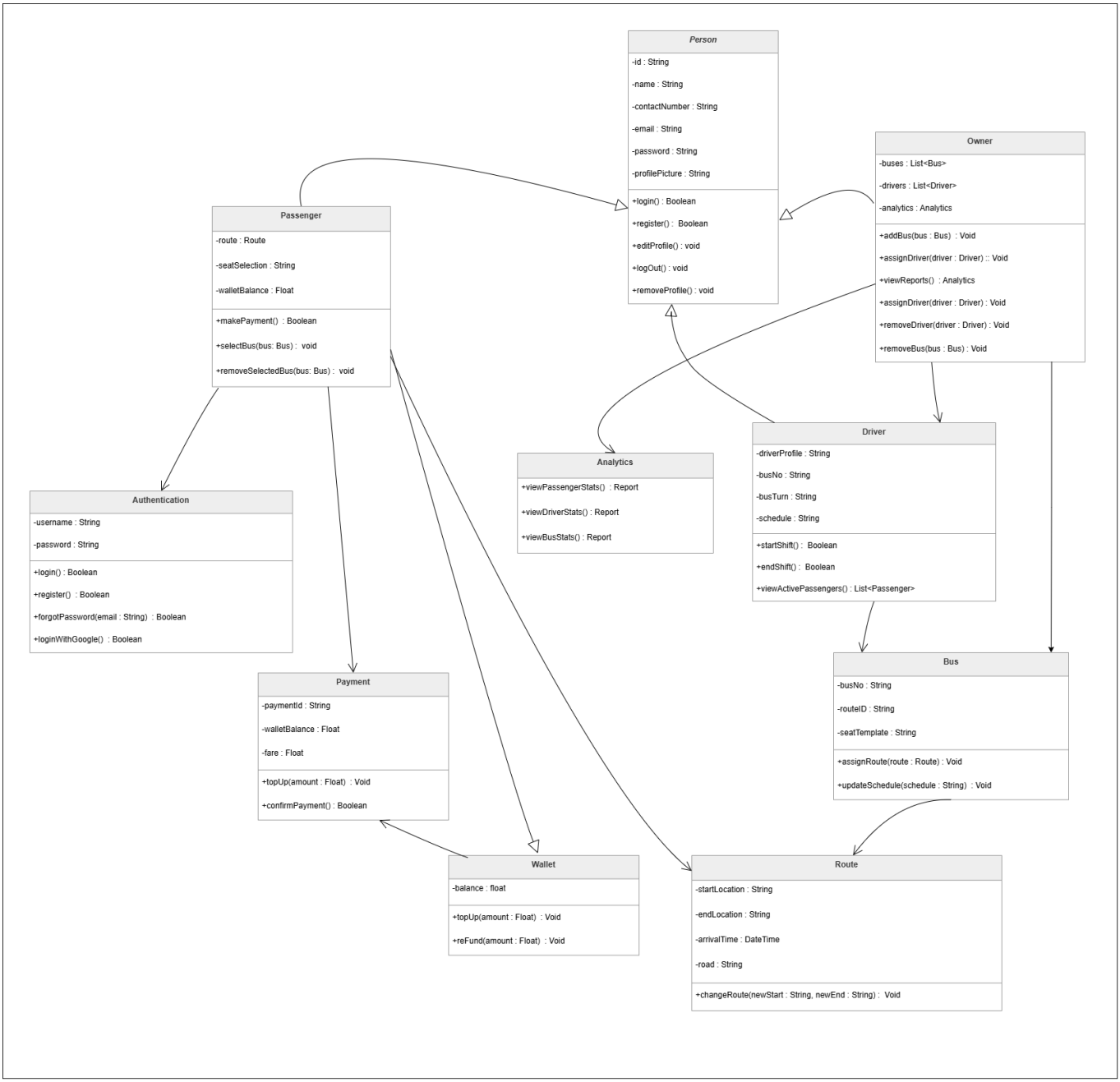

Figure 3: UML Class diagram

6 Sequence Diagram

Link: Sequence Diagram

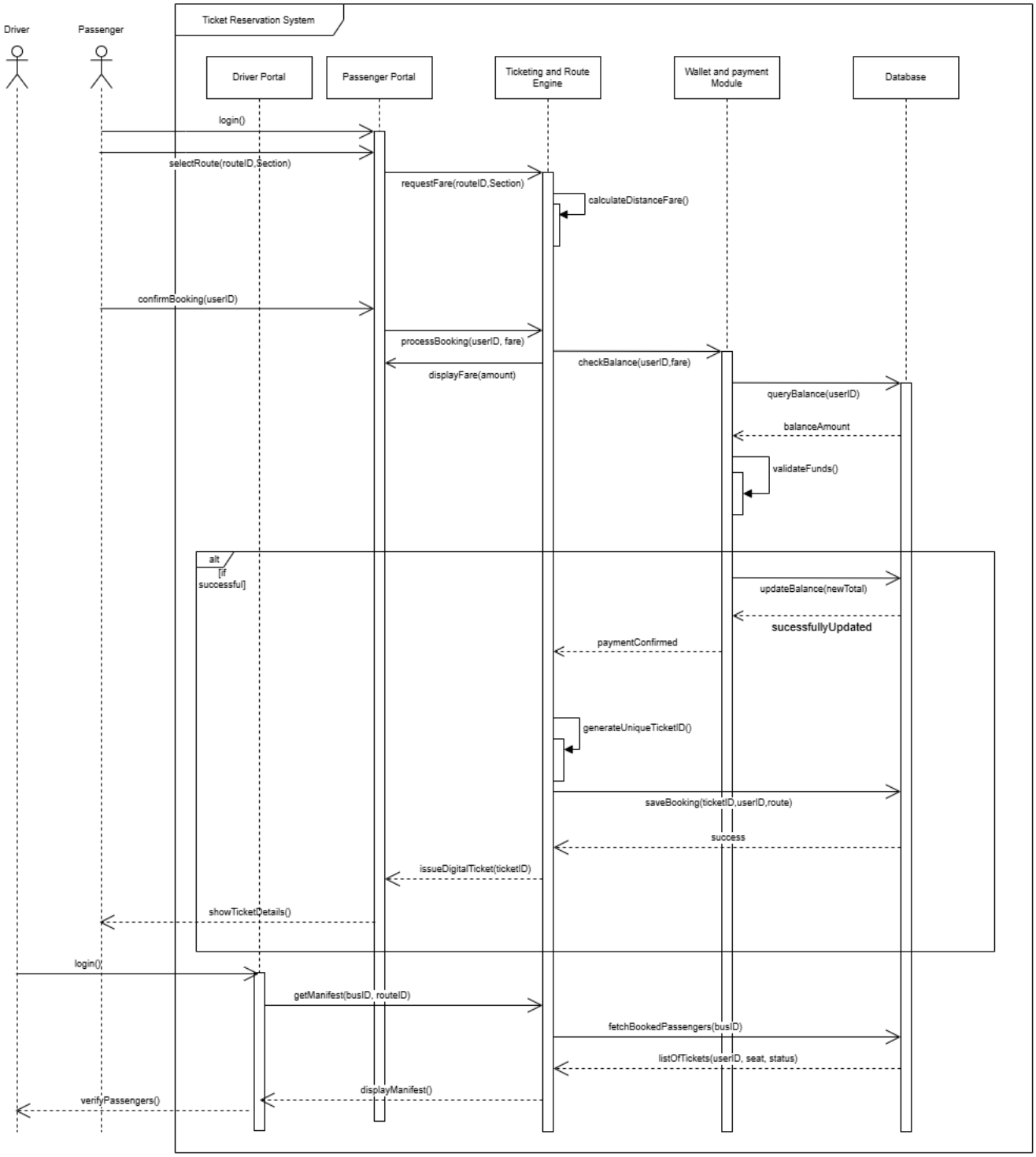

Figure 4: Sequence Diagram

7 Swimlane Diagram

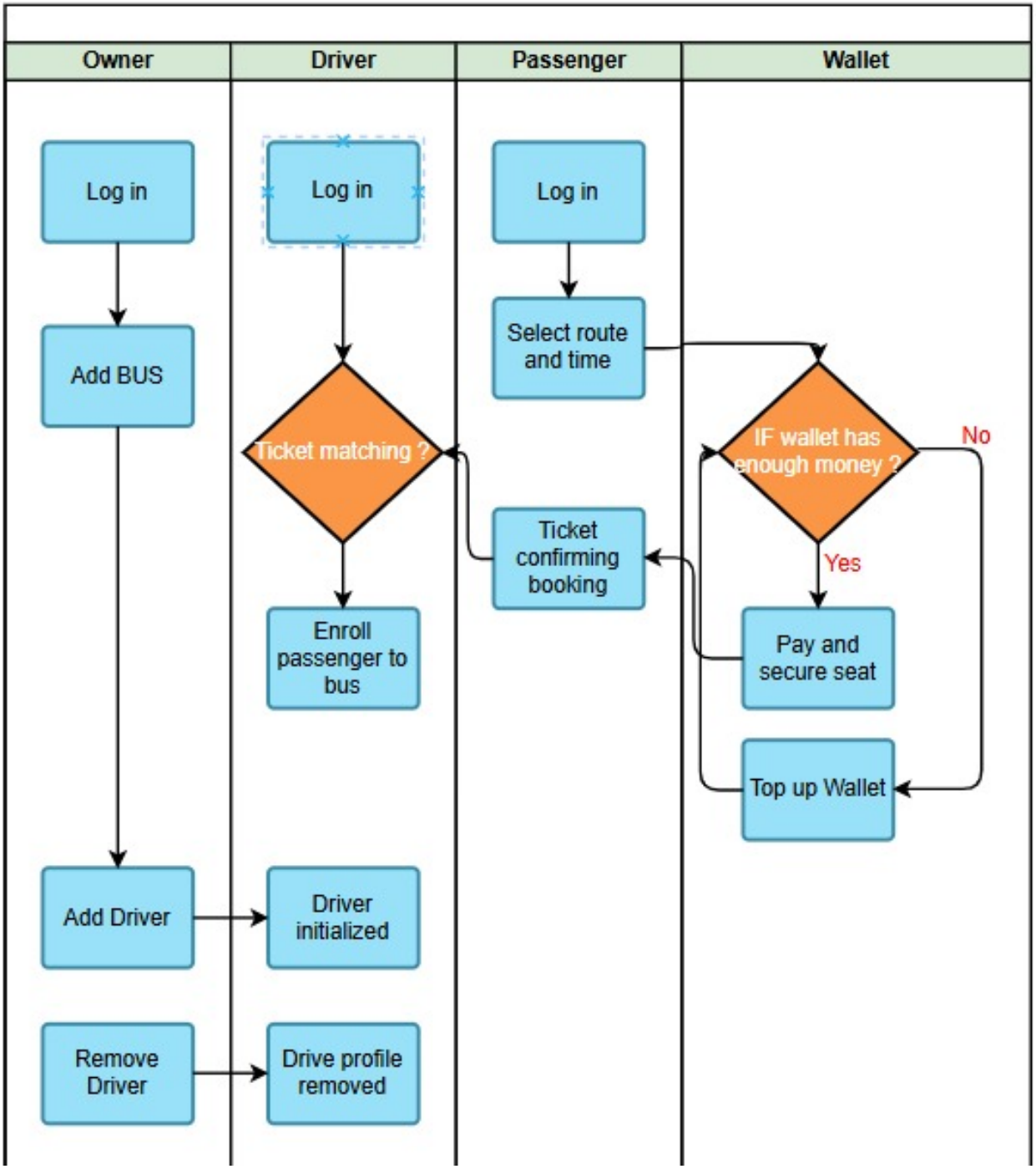

Figure 5: Swimlane Diagram

8 Activity Diagram

Link: Activity Diagram

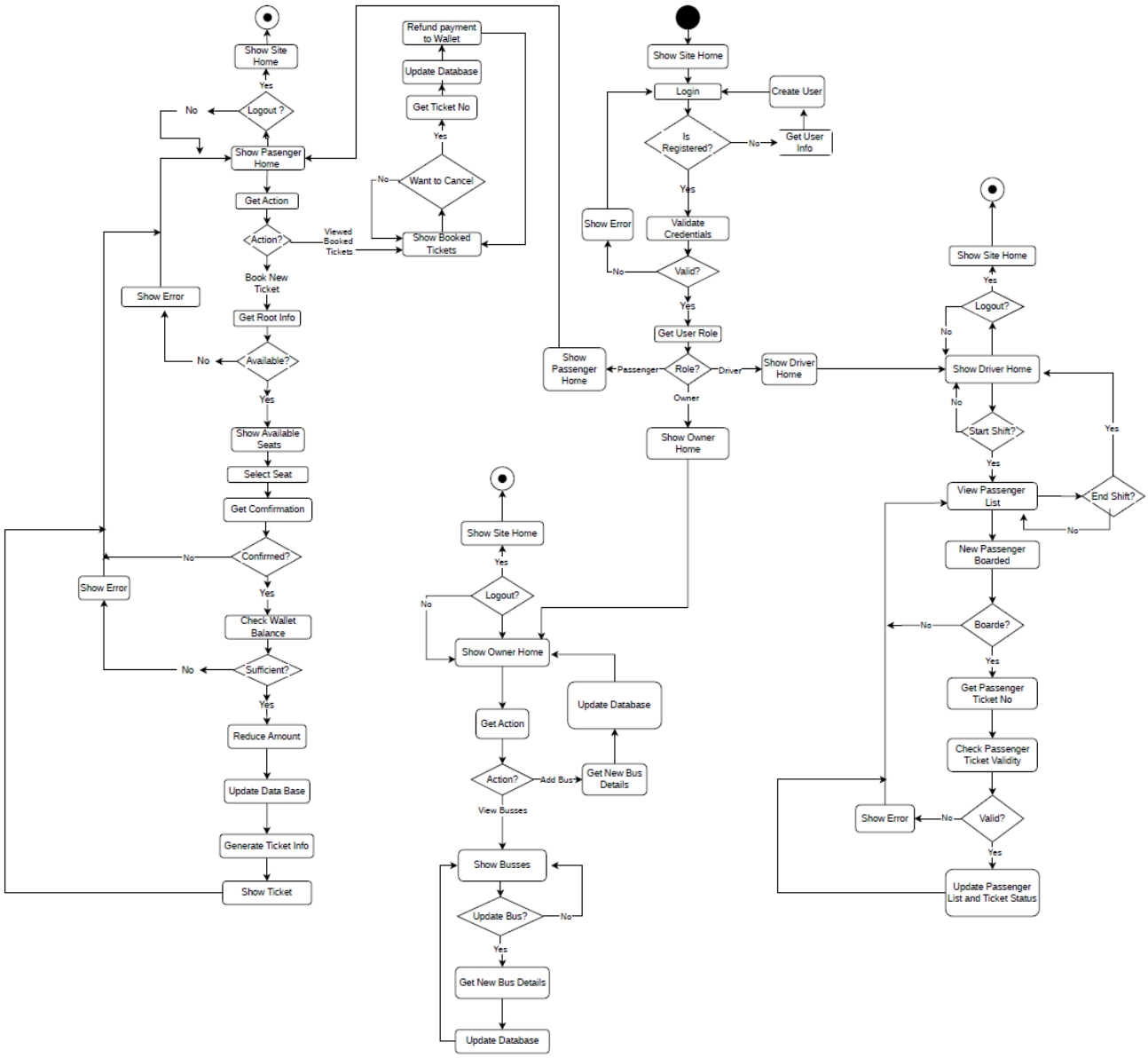

Figure 6: Activity Diagram

9 System Architecture Diagram

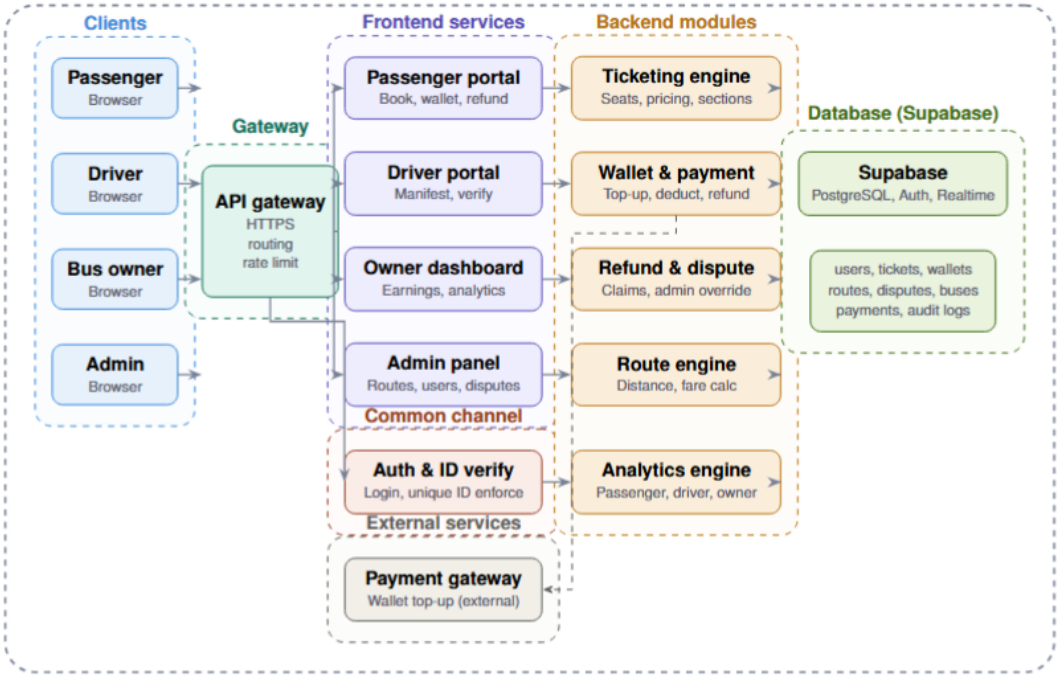

Figure 7: System Architecture Diagram

10 Package Diagram

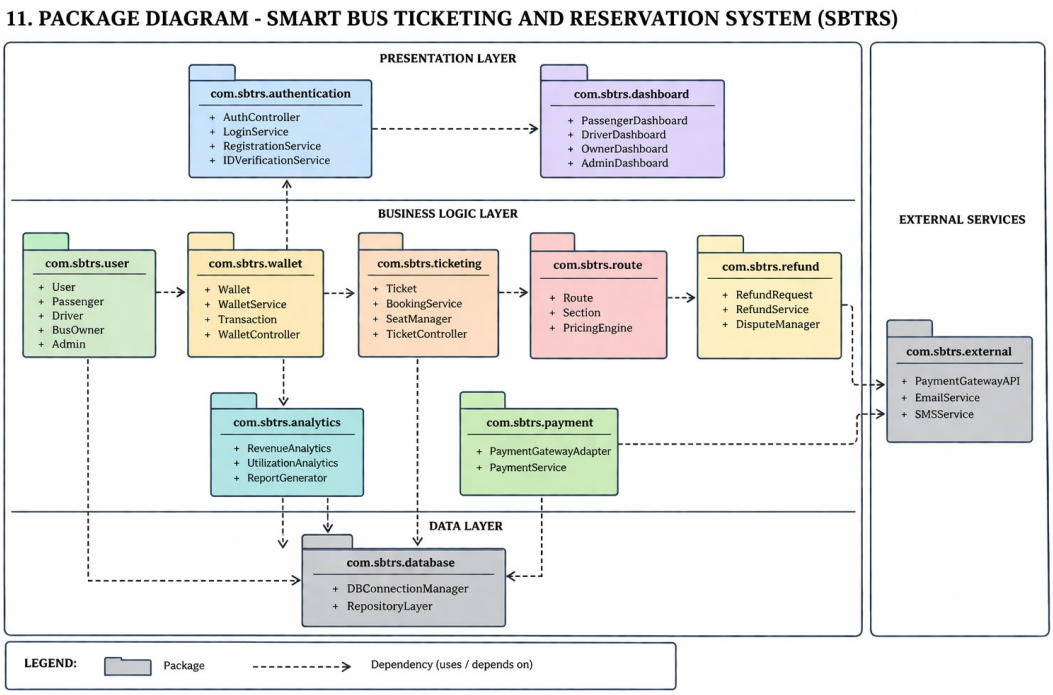

Figure 8: Package Diagram

11 Deployment Diagram

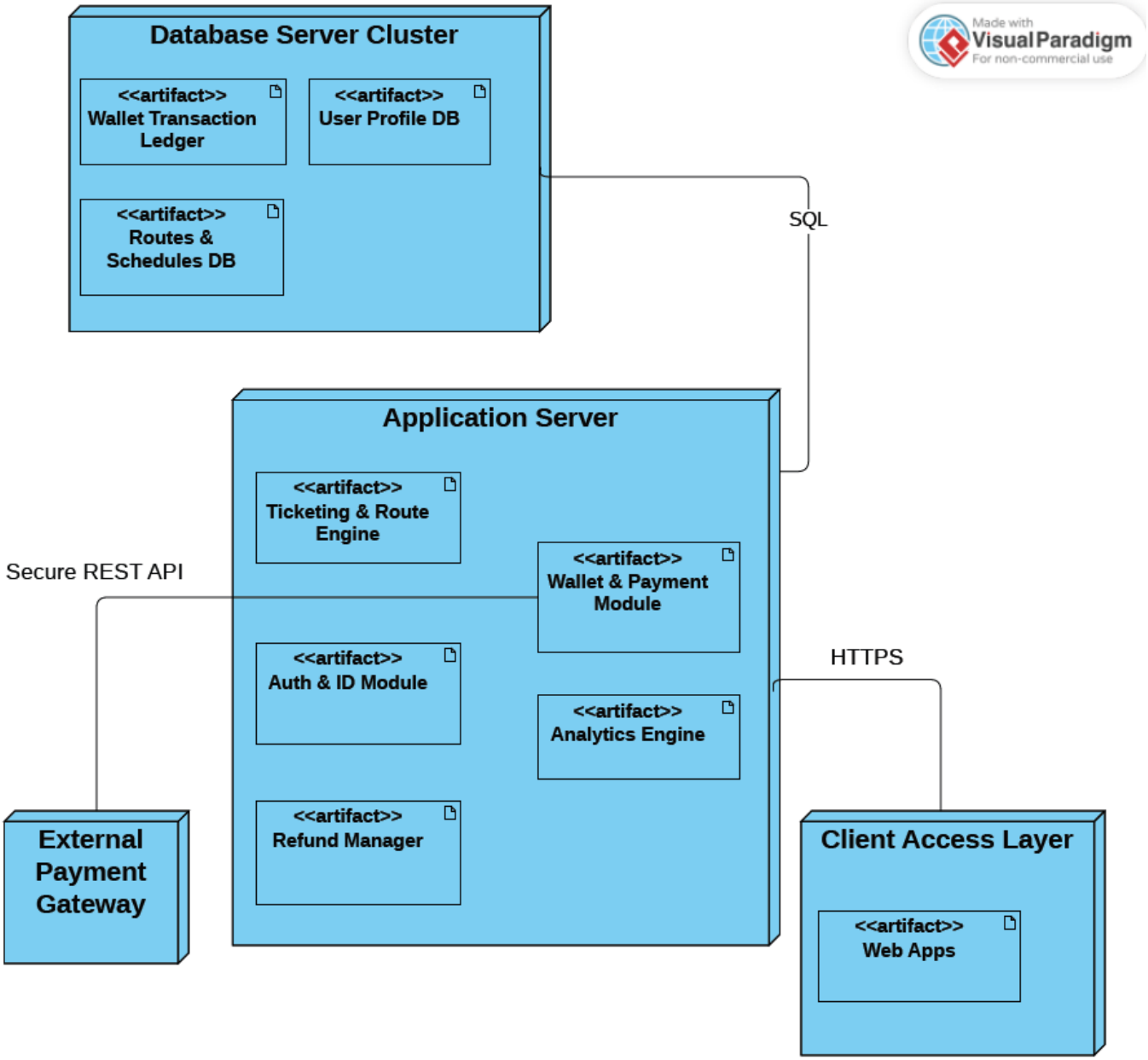

Figure 9: Deployment Diagram

12 Graphical User Interface (GUI) Plan

12.1 Page Structure and Routing

Route	Component	Description
/	HomePage	Search form, featured routes, login/register
/search	SearchResults	List of buses for selected route and date
/select-seats/:busId	SeatSelection	Interactive seat map for booking
/payment	PaymentPage	Payment form and booking summary
/confirmation/:bookingId	ConfirmationPage	Ticket details with QR code
/dashboard	UserDashboard	Upcoming/past bookings and user profile
/admin	AdminPanel	Admin interface for managing routes and users
/profile	ProfilePage	User profile management (personal info, preferences)
/drivers	DriverPanel	Driver-specific dashboard and trip assignments
/owners	OwnerPanel	Bus owners' management interface for fleet and schedules
/wallet	WalletPage	Digital wallet for payments, refunds, and balance tracking
/about	AboutPage	Information about the company and system
/feedback	FeedbackPage	User feedback submission and support contact

Table 2: Page Routing and Component Mapping

12.2 Styling and Responsiveness

- Tailwind CSS for layout, spacing, and typography
- Responsive design using `sm`, `md`, `lg` breakpoints
- Accessibility features: keyboard navigation, alt text

12.3 Development Tools and Libraries

- **Core**: React, React Router DOM, Tailwind CSS, Vite
- **UI Enhancements**: Headless UI, Material UI, Framer Motion, GSAP
- **Deployment**: Vercel or Netlify

Link: Figma Design

13 Test and Deployment Plan

13.1 Testing Strategy

- **Unit Testing:** Individual backend modules will be tested in isolation. For example, the Owner module backend, unit tests must verify the correct calculation of earnings and utilization metrics before they are passed to the frontend.
- **Integration Testing:** Verifies the communication between the Frontend services via the API gateway to the Backend modules and the Supabase database. Critical paths include testing the secure REST API connections to the External Payment Gateway.
- **User Acceptance Testing (UAT):** Stakeholders will validate the role-based dashboards against user and business requirements.
- **Load and Stress Testing:** Load testing will simulate concurrent users booking tickets and topping up wallets to ensure the system can handle island-wide deployment.

13.2 Deployment Pipeline

The deployment pipeline outlines the transition of the STAR system from the Git Repository to the production environment, matching the above defined deployment architecture.

1. **Environment Provisioning**
   - **Database Server Cluster:** Initialize the Supabase environment, configuring separate logical databases for the Wallet Transaction Ledger, User Profile DB, and Routes & Schedules DB.
   - **Application Server:** Provision the hosting environment for the core backend microservices (Ticketing & Route Engine, Wallet & Payment Module, Auth & ID Module, Analytics Engine, Refund Manager).
   - **Client Access Layer:** Set up the web hosting for the browser-based client applications (Passenger, Driver, Bus Owner, Admin).
2. **CI/CD Integration**
   - Code merged from individual teams into the `dev` branch triggers the continuous integration (CI) pipeline, running automated unit and integration tests to validate the combined codebase.
   - The release code is merged into the `main` branch, after successful QA validation in the `dev` environment.
   - Merging into the `main` branch triggers the continuous deployment (CD) production pipeline.
   - Backend services are built and deployed to the production Application Server over secure protocols.
   - Frontend assets are compiled for production and pushed to the Client Access Layer.
3. **Configuration**
   - Securely add the API keys to the production server to activate the Payment Gateway, Email, and SMS integrations.
4. **Sanity Checks and Launch**
   - Execute a final round of automated tests in the production environment.

- Verify HTTPS connectivity for the web apps.
- Monitor the application for the first 7 days to quickly address any payment gateway failures or sudden scalability issues.

5. **Rollback Procedure**

   - If a critical failure occurs during deployment, the CI/CD pipeline will be instructed to redeploy the previous stable tag from the Git repository, and the Supabase database will be restored from the most recent pre-deployment snapshot.

13.3 Test Cases

Test ID ***Module***	**Description**	**Executing Process**	**Expected Result**
TC-01 *Wallet*	Verify prepaid wallet top-up.	1. Log in as Passenger. 2. Navigate to Wallet. 3. Initiate top-up via payment gateway.	Wallet balance accurately reflects the added funds; transaction logged in Wallet Transaction Ledger.
TC-02 *Ticketing*	Prevent duplicate bookings.	1. Passenger selects route and seat. 2. Attempt to book a seat already marked as reserved.	System rejects the booking and prompts the user to select an available seat.
TC-03 *Owner*	Validate Owner Analytics data.	1. Process mock ticket sales for a specific Bus Owner. 2. Query the Analytics Engine.	The backend accurately aggregates earnings and utilization statistics for the Owner Dashboard.
TC-04 *Refund*	Process automated refund.	1. Passenger submits valid refund request. 2. Wait for system processing.	Funds are automatically returned to the Passenger's digital wallet.
TC-05 *Driver*	Verify passenger manifest.	1. Log in as Driver. 2. Access active route schedule.	Driver portal accurately displays all passengers with unique ID bookings for that trip.
TC-06 *Integration*	Verify Backend to Supabase Database connection.	1. Send a POST request via the API to create a new route. 2. Query the Supabase database directly. 3. Send a GET request to fetch the new route.	Data is successfully written to the database without errors, and the exact data is retrieved by the frontend services.
TC-07 *Integration*	Verify secure API connection to External Payment Gateway.	1. Initiate ticket purchase. 2. Trigger Payment Gateway. 3. Monitor API request and response payload.	Valid payment token is received, and the Wallet Transaction Ledger updates successfully without data loss.
TC-08 *Stress*	Validate system stability under high concurrent load.	1. Configure load testing tool. 2. Simulate 50 concurrent users booking the same route simultaneously. 3. Monitor server resource usage.	System maintains 99.9% uptime, processes all requests without crashing, and correctly handles seat booking.

Table 3: Core System Test Cases

14 Git Repository

Link: STAR - Smart Ticketing and Reservation system for buses

15 Potential risks during the project.

- Requirement changes from client company.
- Payment gateway failures due to external payment gateway usage.
- Scalability issues with modules.
- Dependency and compatibility issues in integration stage.
- Since this is a multi-team project, version control might be messy and complex.

16 Actions to Minimize Those Risks

- Agile iterations to minimize overloading the system with more feature based on requirements.
- Constant testing and rigid architecture for bridge with payment gateway.
- Maintaining a shared API contract from the beginning to minimize any potential issues at integration.
- Load testing.
- Maintaining branches as **main**, **dev** and another 8 branches for all 8 teams so that all commits follow an overseeing of group leaders before being merged to the dev branch. Only at deployment stage it would be merged to the main branch.

ANNEXURE

Index	Name	Contribution
230212H	Gunasekara L.U.A.	Swimlane Diagram + Overall documentation
230470U	Peiris T.S.R.	System Architecture Diagram
230256U	Ilankoon I.M.M.K.B.	System Architecture Diagram
230258D	Imaduwage O.N.H.	System Architecture Diagram
230259G	Imbulpitiya B.N.	Swimlane Diagram
230033J	Amarasinghe A.A.D.K.	ER Diagram + SRS documentation
230082G	Bandara W.D.A.C.	ER Diagram + SRS documentation
230164K	Dissanayake R.K.T.	ER Diagram
230629R	Tennakoon U.G.R.B.	ER Diagram
230086X	Basnayaka B.A.L.G.T.	Package diagram
230103B	Charuka A.A.K.	Package diagram
230191N	Gajasinghe G.M.C.R.	Package diagram
230182M	Fernando K.S.N.G.	Package diagram
230306B	P.G.I. Jithmal	Deployment diagram
230442L	W.N.A. Chanul Nimadith	Deployment diagram
230300C	Jayaweera N.S.	Deployment diagram
230203G	Gunarathna K.T.M.B.	Deployment diagram
230294H	Jayawardana K.P.P.	Deployment diagram
230566U	S.M.R.R. Samarasinghe	Test and Deployment plan
230458P	H.H. Palihena	Test and Deployment plan
230493R	N.P.P. Piyumal	Test and Deployment plan
230321P	A.H.D. Karunanayake	Test and Deployment plan
230613M	M.N.N. Shehan	Test and Deployment plan
230532M	Ranmuthu S.G.	Activity Diagram
230480B	Perera K.S.D.	Activity Diagram
230627J	Suraweera S.C.	Activity Diagram
230296P	Jayawardana P.H.N.D.	Activity Diagram
230071X	Balasooriya U.B.M.T.S.B.	Activity Diagram
230162D	L.M. Dissanayake	Sequence Diagram
230560V	H.S. Samahon	Sequence Diagram
230282U	M.W. Jayasinghe	Sequence Diagram
230093P	G.M.D.C. Buddhadasa	Sequence Diagram
230287N	J.M.T.R.B. Jayasundara	Sequence Diagram
230526A	Ranawaka R.A.C.D.	Gantt chart
230045X	Anthony C.S.B.	Gantt chart
230697V	Weerasinghe J.A.H.R.	Gantt chart
230121D	De Mel D.J.	Gantt chart

Table 4: Team Members and Contributions

- All leaders mutually corresponded in distributing the workload and task allocation.
- The GUI plan was designed with mutual collaboration of the front end teams.